

JingleEat

Твой новогодний шеф-повар с ИИ

Swift 6 | iOS 26 | Hybrid UI (UIKit + SwiftUI)

Презентация финальной защиты

Описание основного функционала



Smart Video Feed

Вертикальная лента рецептов с автоплеем. Реализована на AVFoundation с бесшовным перелистыванием и кешированием видео-файлов.



AI Sous-Chef

Интеграция с GigaChat API для подбора рецептов. Использует контекстные промпты для генерации кулинарных инструкций.



Личный кабинет

Просмотр сохраненного контента и статистики. Безопасная авторизация с хранением сессии в Apple Keychain.

Схематичная карта экранов



Навигация: *UITabBarController (UIKit)* в качестве корневого контейнера + *UIHostingController* для *SwiftUI Views*.

Схема зависимостей компонентов

Presentation Layer

- ✓ Views (SwiftUI / UIKit)
- ✓ ViewModels (@StateObject)
- ✓ VideoPlayerManager (Shared Instance)

Domain & Data Layer

- ✓ GigaChatService (OAuth 2.0 Flow)
- ✓ VideoLoader (FileManager Cache)
- ✓ KeychainService (Security API)

Интеграция GigaChat API

Работа с запросами

Реализован асинхронный флоу получения Access Token и последующей отправки промптов через `URLSession`.

Используется `JSONDecoder` для обработки структурированных ответов от модели.

```
func fetchToken() async throws {  
    var request = URLRequest(url: authURL)  
    request.httpMethod = "POST"  
    request.allHTTPHeaderFields = authHeaders  
  
    let (data, _) = try await  
        URLSession.shared.data(for: request)  
    // Обработка токена...  
}
```


Оптимизация видео-ленты

Управление жизненным циклом загрузки

Использование Structured Concurrency. При быстрой прокрутке текущая задача загрузки отменяется, чтобы не забивать поток и канал связи.

```
func handleScrollChange(newID: String?) {
    currentDownloadTask?.cancel()
    currentDownloadTask = Task {
        let videoURL = try await VideoLoader.shared.loadVideo(id: newID)
        // Воспроизведение...
    }
}
```

Безопасность: Apple Keychain

Secure Data Storage

Для хранения паролей и токенов сессии используется Keychain API. Это обеспечивает защиту данных на уровне системы.

- Шифрование данных
- Хранение в Secure Enclave
- Сохранение сессии при переустановке

```
let query: [String: Any] = [  
    kSecClass: kSecClassGenericPassword,  
    kSecAttrAccount: "user_token",  
    kSecValueData: tokenData  
]  
SecItemAdd(query as CFDictionary, nil)
```

Проблемы и их решения

Сложность	Как преодолели?
Лаги при скролле видео	Внедрение <code>VideoPlayerManager</code> и локальное кеширование в <code>Caches/Video</code> .
Смешивание UIKit и SwiftUI	Использование <code>UIHostingController</code> для интеграции экранов SwiftUI в TabBar.
Безопасность данных	Реализация <code>KeychainService</code> для зашифрованного хранения токенов.
Асинхронные утечки	Применение <code>cancel()</code> для Task при деинициализации ViewModels.

Compliance & Теория кода

Принципы разработки

`</>` **SOLID**: Разделение ответственности между сервисами.

🛡️ **Thread Safety**: UI обновляется строго на `@MainActor`.

UX Стандарты

🌙 **Dark Mode**: Полная поддержка темной темы.

🔧 **Testing**: Unit-тесты для маппинга JSON-моделей.

Демонстрация

Live-демо на симуляторе iOS

